

diseases.json

Document Statistics:

- File Size: 941 bytes
- Total Keys: 30
- Total Values: 20
- Max Depth: 2
- Complexity: Simple
- Processing: standard

Hypertension:

icd_code:

"I10"

structure:

"I (Circulatory) + 10 (Essential)"

Type 2 Diabetes:

icd_code:

"E11.9"

structure:

"E11 (Endocrine) + .9"

Ischemic Heart Disease:

icd_code:

"I25.9"

structure:

"I25 (Chronic IHD) + .9"

COPD:

icd_code:

"J44.9"

structure:

"J44 (Respiratory)"

Urinary Tract Infection:

icd_code:

"N39.0"

structure:

"N39 (Urinary)"

Low Back Pain:

icd_code:

"M54.50"

structure:

"M54.5 (Low Back)"

Joint Pain:

icd_code:

"M25.50"

structure:

"M25.5 (Joint)"

Common Cold:

icd_code:

"J00"

structure:

"Acute Nasopharyngitis"

General Body Ache:

icd_code:

"R52"

structure:

"Pain NEC"

Cosmetic Surgery:

icd_code:

"Z41.1"

structure:

"Elective Cosmetic"

policies.json

Document Statistics:

- File Size: 3,822 bytes
- Total Keys: 84
- Total Values: 78
- Max Depth: 3
- Complexity: Simple
- Processing: standard

Ayushman Bharat (PM-JAY):

coverage:

"■5 Lakh + ■5 Lakh (70+)"

type:

"Government"

eligibility:

"Rural poor, urban workers, and all citizens above 70"

rule:

"Covers hospitalization, pre-existing diseases from Day 1. 24-hour admission required. Includes pre/post hospitalization."

allowed_diseases:

[0]:

"Hypertension"

[1]:

"Type 2 Diabetes"

[2]:

"Ischemic Heart Disease"

[3]:

"COPD"

review_diseases:

[0]:

"Urinary Tract Infection"

[1]:

"Low Back Pain"

[2]:

"Joint Pain"

ESIC:

coverage:

"Unlimited"

type:

"Government"

eligibility:

"Employees earning ≤ ■21,000/month (■25k for PWD)"

rule:

"Full medical care with contribution. Includes wage benefits. Requires employment-based eligibility."

allowed_diseases:

[0]:

"Hypertension"

[1]:

"Type 2 Diabetes"

[2]:

"Common Cold"

review_diseases:

[0]:

"COPD"

CGHS:

coverage:

"Up to ₹10 Lakh"

type:

"Government"

eligibility:

"Central Govt employees & pensioners"

rule:

"Referral-based system. Includes AYUSH and Allopathy."

allowed_diseases:

[0]:

"Hypertension"

[1]:

"COPD"

review_diseases:

[0]:

"Ischemic Heart Disease"

PMSBY:

coverage:

"₹2 Lakh"

type:

"Government"

eligibility:

"18–70 years with bank account"

rule:

"Covers accidental death/disability only. No illness coverage."

allowed_diseases:

[]

review_diseases:

[]

PMJJBY:

coverage:

"₹2 Lakh"

type:

"Government"

eligibility:

"18–50 years"

rule:

"Life insurance only. Covers death, not hospitalization."

allowed_diseases:

[]

review_diseases:

[]

UHS:

coverage:

"₹30,000"

type:

"Government"

eligibility:

"BPL/APL families (5–70 years)"

rule:

"Includes hospitalization + maternity + wage loss benefits."

allowed_diseases:

[0]:

"Common Cold"

[1]:

"Urinary Tract Infection"

review_diseases:

[0]:

"Hypertension"

AABY:

coverage:

"■75,000"

type:

"Government"

eligibility:

"Rural landless workers (18–59)"

rule:

"Covers death and disability only. No medical expenses."

allowed_diseases:

[]

review_diseases:

[]

HDFC ERGO Optima Secure:

coverage:

"■5 Lakh – ■2 Crore"

type:

"Private"

eligibility:

"All ages"

rule:

"4X coverage benefit. Covers consumables. Waiting period applies."

allowed_diseases:

[0]:

"COPD"

[1]:

"Ischemic Heart Disease"

review_diseases:

[0]:

"Hypertension"

Star Comprehensive:

coverage:

"■5 Lakh – ■1 Crore"

type:

"Private"

eligibility:

"3 months – 65 years"

rule:

"Includes OPD, maternity, newborn care, bariatric surgery."

allowed_diseases:

[0]:

"Hypertension"

[1]:

"Type 2 Diabetes"

review_diseases:

[0]:

"Joint Pain"

Niva Bupa ReAssure 2.0:

coverage:

"■5 Lakh – ■1 Crore"

type:

"Private"

eligibility:

"18–99 years"

rule:

"Unlimited restoration. 2-hour hospitalization coverage."

allowed_diseases:

[0]:

"COPD"

review_diseases:

[0]:

"Hypertension"

Care Supreme:

coverage:

"■3 Lakh – ■1 Crore"

type:

"Private"

eligibility:

"18+"

rule:

"Unlimited recharge. AYUSH & domiciliary included."

allowed_diseases:

[0]:

"Type 2 Diabetes"

review_diseases:

[0]:

"COPD"

Aditya Birla Activ One MAX:

coverage:

"■2 Lakh – ■6 Crore"

type:

"Private"

eligibility:

"18+"

rule:

"Day 1 chronic care coverage. Includes prosthetics & equipment."

allowed_diseases:

[0]:

"Hypertension"

[1]:

"Type 2 Diabetes"

review_diseases:

[0]:

"COPD"

```
from groq import Groq
import os
from dotenv import load_dotenv

load_dotenv()

client = Groq(api_key=os.getenv("GROQ_API_KEY"))

def generate_reason(disease, policies, status):

    policy_text = ", ".join(policies) if policies else "No policies"

    prompt = f"""
    Patient Disease: {disease}
    Policies: {policy_text}
    Decision: {status}

    Give a clear explanation in 5-6 lines:
    - Why approved/rejected
    - Which rule applies
    - Coverage insight
    Keep it professional and concise.
    """

    try:
        response = client.chat.completions.create(
            model="llama-3.1-8b-instant",
            messages=[{"role": "user", "content": prompt}]
        )

        return response.choices[0].message.content

    except Exception as e:
        return f"AI Error: {str(e)}"
```

```
from ai_reason import generate_reason
import streamlit as st
import json
import time
import base64
from PIL import Image

# ----- LOAD DATA -----
with open("data/diseases.json", "r") as f:
    diseases = json.load(f)

with open("data/policies.json", "r") as f:
    policies = json.load(f)

# ----- SAFE LOGO LOADER -----
def load_logo():
    try:
        with open("logo.png", "rb") as f:
            return base64.b64encode(f.read()).decode()
    except:
        return None

logo_base64 = load_logo()

# ----- PAGE CONFIG -----
st.set_page_config(page_title="Healthcare AI", layout="wide")

# ----- UI DESIGN -----
st.markdown(f"""
<style>
.stApp {{
    background: radial-gradient(circle at center, #0f172a 0%, #020617 100%);
    color: #e2e8f0;
}}

/* BACKGROUND LOGO */
.stApp::before {{
    content: "";
    position: fixed;
    top: 50%;
    left: 50%;
    width: 1500px;
    height: 1500px;
    background: url("data:image/png;base64,{logo_base64}") no-repeat center;
    background-size: contain;
    opacity: 0.06;
    transform: translate(-50%, -50%);
    z-index: 0;
}}

/* cards */
.card {{
    background: rgba(15,23,42,0.75);
    padding: 20px;
    border-radius: 12px;
    margin-bottom: 15px;
    backdrop-filter: blur(10px);
}}

/* text colors */
.green {{ color: #22c55e; font-weight: bold; }}
.yellow {{ color: #facc15; font-weight: bold; }}

/* inputs */
input, .stTextInput input, .stNumberInput input {{
    background-color: #1e293b !important;
    color: white !important;
}}

/* button */
.stButton>button {{
    background: #2563eb;
    color: white;
    border-radius: 8px;
    height: 45px;
    width: 100%;
}}

h1,h2,h3 {{ color:#38bdf8; }}

</style>
""", unsafe_allow_html=True)

st.title("AI Healthcare Insurance System")

# ----- SESSION -----
if "submitted" not in st.session_state:
    st.session_state.submitted = False

if "patient_data" not in st.session_state:
    st.session_state.patient_data = {}

# ===== BACK =====
if st.session_state.submitted:
    if st.button("Back"):
        st.session_state.submitted = False
        st.session_state.patient_data = {}
        st.rerun()

# ===== INPUT =====
if not st.session_state.submitted:

    st.subheader("Patient Details")

    c1, c2 = st.columns(2)

    with c1:
        name = st.text_input("Name")
        age = st.number_input("Age", 1, 120)

    with c2:
        gender = st.selectbox("Gender", ["Male", "Female", "Other"])
        disability = st.radio("Physical Disability", ["No", "Yes"])

    st.subheader("Analysis Mode")

    mode = st.radio("", ["Manual Analysis", "Auto Analysis"], horizontal=True)

    if mode == "Manual Analysis":
        disease_input = st.text_input("Enter Disease")
        selected_scheme = st.selectbox("Select Policy", list(policies.keys()))
```



```
else:
    disease_input = st.selectbox("Select Disease", list(diseases.keys()))
    selected_scheme = None

if st.button("Run Analysis"):

    if not name.strip():
        st.error("⚠️ Enter valid name")
        st.stop()

    d_input = disease_input.strip().lower().replace(" ", "")
    disease_map = {k.lower().replace(" ", ""): k for k in diseases.keys()}

    if d_input not in disease_map:
        st.error("⚠️ Invalid Disease")
        st.stop()

    st.session_state.patient_data = {
        "name": name,
        "age": age,
        "gender": gender,
        "disease": disease_map[d_input],
        "mode": mode,
        "scheme": selected_scheme,
        "disability": disability,
    }

    st.session_state.submitted = True
    st.rerun()

# ===== OUTPUT =====
else:

    pdata = st.session_state.patient_data
    disease = pdata["disease"]
    icd = diseases[disease]

    # ----- SUMMARY -----
    st.markdown(f"""
    <div class="card">
    <b>Name:</b> {pdata['name']} <br>
    <b>Age:</b> {pdata['age']} <br>
    <b>Gender:</b> {pdata['gender']} <br>
    <b>Disease:</b> {disease} <br>
    <b>Disability:</b> {pdata['disability']}
    </div>
    """, unsafe_allow_html=True)

    # ----- ICD -----
    st.markdown(f"""
    <div class="card">
    <b>ICD Code:</b> {icd['icd_code']} <br>
    <b>Structure:</b> {icd['structure']}
    </div>
    """, unsafe_allow_html=True)

    # ----- LOGIC -----
    def get_status(pol):
        if disease in pol.get("allowed_diseases", []):
            return "Approved"
        elif disease in pol.get("review_diseases", []):
            return "Review"
        return "Rejected"

    approved, review = [], []

    if pdata["mode"] == "Auto Analysis":
        for name, pol in policies.items():
            status = get_status(pol)
            if status == "Approved":
                approved.append(name)
            elif status == "Review":
                review.append(name)
    else:
        pol = policies[pdata["scheme"]]
        status = get_status(pol)
        if status == "Approved":
            approved.append(pdata["scheme"])
        elif status == "Review":
            review.append(pdata["scheme"])

    st.subheader("Insurance Decision")

    # ----- FULL REJECTION -----
    if not approved and not review:

        st.error("⚠️ Not Covered by Any Policy")

        with st.spinner("AI analyzing..."):
            time.sleep(1)

        ai_output = generate_reason(disease, [], "Rejected")

        st.markdown(f"""
        <div class="card">
        <b>AI Reason:</b><br>{ai_output}
        </div>
        """, unsafe_allow_html=True)

        st.stop()

    # ----- APPROVED -----
    if approved:
        st.success(f"Approved Policies: {approved}")

    if review:
        st.warning(f"Under Review: {review}")

    # ----- AI AUTO -----
    with st.spinner("Generating AI explanation..."):
        time.sleep(1)

    final_status = "Approved" if approved else "Review"
    ai_output = generate_reason(disease, approved + review, final_status)

    st.markdown(f"""
    <div class="card">
    <b>AI Explanation:</b><br>{ai_output}
    </div>
    """, unsafe_allow_html=True)
```

```
import os
from dotenv import load_dotenv
from crewai import Agent, Task, Crew

load_dotenv()

os.environ["OPENAI_API_KEY"] = os.getenv("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")
os.environ["OPENAI_API_BASE"] = "https://openrouter.ai/api/v1"

policy_agent = Agent(
    role="Policy Expert",
    goal="Check insurance coverage",
    verbose=True
)

decision_agent = Agent(
    role="Decision Explainer",
    goal="Explain decision clearly",
    verbose=True
)

def run_agents(disease, scheme, status):

    task = Task(
        description=f"Explain why {disease} is {status} under {scheme}",
        agent=decision_agent
    )

    crew = Crew(
        agents=[policy_agent, decision_agent],
        tasks=[task],
        verbose=True
    )

    return crew.kickoff()
```

```
import pandas as pd
import json

icd_data = pd.read_csv("data/icd_codes.csv")

with open("data/policies.json") as f:
    policies = json.load(f)

def process_claim(disease, scheme):
    disease = disease.strip().lower()

    # ICD match
    match = icd_data[icd_data["Disease"].str.lower() == disease]
    icd_code = match.iloc[0]["ICD_Code"] if not match.empty else "Unknown"

    scheme_data = policies["schemes"].get(scheme, {})

    # Convert lists to lowercase
    approved = [d.lower() for d in scheme_data.get("approved", [])]
    review = [d.lower() for d in scheme_data.get("review", [])]
    rejected = [d.lower() for d in scheme_data.get("rejected", [])]

    if disease in approved:
        status = "Approved"
    elif disease in review:
        status = "Needs Review"
    elif disease in rejected:
        status = "Rejected"
    else:
        status = "Unknown"

    amount = scheme_data.get("coverage_amount", 0)

    return icd_code, status, amount
```

